

ECOM EXPERTS

Ecom healthcorner

Manual de usuario, técnico y de desarrollo

ecom_healthcorner · 08/09/2026

Contenido

1. Puesta en marcha (orden recomendado)

2. Negocios

3. Productos por negocio

4. Máquinas y reponedores

5. El huésped: comprar y recoger

6. Panel del hotel

7. Pedidos, albaranes y liquidaciones

8. RGPD

9. Modo quiosco

10. Problemas frecuentes

1. Estructura

Clases (classes/)

Admin controllers

Front controllers y rutas (hookModuleRoutes)

2. Tablas (15)

3. Motor de precios (híbrido, multi-versión)

4. Pago

5. Vending (doble stock)

6. Aislamiento y seguridad

7. i18n y correos

8. Configuración (claves ECOMHC_*)

Sintaxis PHP: suelo 7.1

Métodos eliminados en 9.1 (Fatal) → sustitutos

Firmas que cambian

Motor de precios (HÍBRIDO — decisión del usuario)

Carrier oculto

Checkout / pedido

Admin / tabs

Front layout propio

Traducciones

Cifrado / Redsys / QR

DB / instalador

1. Instalar / actualizar / desinstalar

2. Depuración

3. Añadir un formato de planograma

4. Añadir un método de pago

5. Reglas de firma Redsys (crítico)

6. API propia (para la máquina)

7. Webservice nativo

8. App del reponedor

9. Convenciones

10. Puntos abiertos

API 1 — Webservice nativo de PrestaShop

Activación

Autenticación

Sintaxis común

Recursos publicados por Health Corner

Ejemplos

API 2 — API propia de Health Corner (máquina de vending)

Por qué existe

URL base

Autenticación

Reglas comunes

Formato de respuesta

Endpoints

Ejemplos

Cómo integrar la máquina según lo que permita JSuite

Seguridad

Health Corner convierte tu PrestaShop en una red de **mini-tiendas de parafarmacia por negocio** (hoteles, posadas, gimnasios, apartamentos...), aisladas de la tienda normal, con pago propio, código de recogida, panel para cada hotel y app para el reponedor de las máquinas de vending.

Menú del back office: *Health Corner* (menú propio) → Negocios · Productos por negocio · Precios · Máquinas vending · Pedidos HC · Liquidaciones · API y registro · Configuración · **Ayuda** · Logs. La página **Ayuda** muestra en todo momento las URLs actuales de cada negocio, del panel, de la app del reponedor, de la API y del cron, más un semáforo de estado.

1. Puesta en marcha (orden recomendado)

1. **Configuración → Pago (Redsys):** entorno (test/producción), FUC, terminal y **clave** (escríbela y guárdala en ESTE servidor; si migras la tienda, vuelve a guardarla). Activa **Bizum / Apple Pay / Google Pay** con el campo *Métodos de pago* y la **tokenización** si tendrás estancias largas. Marca las **otras formas de pago** (PayPal, transferencia...) que quieras ofrecer.
2. **Configuración → Checkout y emails:** remitente, DNI/teléfono obligatorios...
3. **Configuración → Máquinas y planograma:** modo de integración (A telemetría API / B ficheros / **C conteo manual**, el de serie), umbral de seguridad, email y WhatsApp del reponedor para las alertas.
4. **Configuración → Aislamiento:** activa el bloqueo por negocio si quieres que quien entra por la URL de un hotel no pueda navegar la tienda normal; añade las **IPs** de tu personal.
5. **Negocios:** crea cada hotel/negocio. 6. **Productos por negocio:** asigna el catálogo. 7. **Máquinas:** da de alta máquinas y reponedores. 8. Comparte el **QR/URL** de cada negocio.

Tras subir una versión nueva: entra en **Configuración** (limpia caché y re-registra rutas/hooks) y regenera el `.htaccess` en *Preferencias → SEO y URLs*.

2. Negocios

En **Health Corner → Negocios**, "Nuevo": nombre, slug (URL), **tipo de negocio** (hotel, posada, gimnasio...) y **etiqueta de la unidad** (Habitación, Apartamento, Taquilla, Mesa...). Datos del comercial (razón social, NIF, email, teléfono, dirección), **branding** (logo con vista previa, colores) y ajustes de servicio (máquina sí/no, ubicación, horario, plazo, mínimo de pedido, % recargo, % comisión).

Al crear un negocio se generan automáticamente: su **tienda** (`/hc/{slug}`), su **cliente comercial** y sus **hojas legales** en CMS → *categoría Hoteles*. Con "**Ver**" obtienes la tienda, el QR (copiar/descargar/compartir), **QR con UTM** por ubicación, el **informe de ventas por origen** y los **usuarios del panel** del hotel.

3. Productos por negocio

Elige el negocio y asigna productos (buscador o por categoría). Por cada uno marcas **Máquina / Entrega / Destacado / Activo**, el **canal/espiral**, el **stock de máquina** y la **posición**. Las cabeceras "(todos)" marcan/desmarcan toda una columna. Botón **Guardar todo**. Puedes **copiar** el catálogo de otro negocio, **aplicar a todos**, hacer **redondeo comercial** (,95 / ,50 / entero) y **exportar/importar CSV** con **vista previa**.

El precio por negocio se resuelve solo (recargo %, precio fijo o hereda el PVP). Los productos solo de **máquina** se ven pero **no se pueden añadir al carrito**: son de consulta.

4. Máquinas y reponedores

En **Máquinas vending** das de alta cada máquina y, dentro, la **reposición con conteo real**: indicas lo que quedaba y lo que cargas; la diferencia se registra como ventas. Exportas el **planograma** (genérico, JSuite/Jofemar o telemetría).

Reponedores: en la misma pantalla das de alta a **clientes de la tienda** por su email (acceso a todos los negocios o a uno). Entran en la **app del reponedor**:

```
https://TU-TIENDA/hc/reponedor
```

Login con email+contraseña de cliente → **negocio** → **máquina** → **buscan** o **escanean** el código de barras → suben/bajan o fijan el stock con botones grandes. Instalable como app (PWA). Cada cambio queda registrado. Si el stock cae bajo el umbral, el reponedor recibe un **correo de aviso** (con enlace de WhatsApp), sin avisos repetidos ni barrido de histórico.

5. El huésped: comprar y recoger

Abre el QR/URL del negocio, ve la tienda con la marca del hotel (buscador con desplegable, 24h/entrega y "disponible en la máquina"), añade al carrito con popup tipo PrestaShop, rellena sus datos (número de habitación/unidad) y paga con tarjeta, Bizum, etc. Recibe:

- Un **código de recogida** y un **QR** en pantalla y por **email** (con desglose, en su idioma — detectado por el navegador).
- Una **página de seguimiento** `/hc/{slug}/track?c=CÓDIGO` con el estado (pagado → preparando → listo → entregado).

El personal confirma la entrega **escaneando el QR** o tecleando el código en el **Panel del hotel**. Con pagos no inmediatos (transferencia) el pedido queda "en espera de pago" y no avanza hasta que lo cobres.

6. Panel del hotel

```
https://TU-TIENDA/hc/{slug}/panel
```

Entra con un **cliente de PrestaShop** mapeado (en *Negocios* → *Ver* → *Usuarios del panel*). El login del panel cuenta con un diseño premium adaptativo tipo tarjeta que utiliza el logotipo y los colores corporativos de cada hotel para una personalización total, incluyendo un botón para mostrar/ocultar la contraseña.

Una vez dentro, el usuario ve **solo de su negocio**: pedidos (y confirmar entregas con escáner), productos, stock de máquina y **métricas** (ticket medio, por canal, origen de QR, ventas por franja horaria, surtido por momento del día), más su QR y enlace.

7. Pedidos, albaranes y liquidaciones

- **Pedidos HC**: filtros, export CSV, **albarán por pedido** y **albarán de reparto** (hoja de preparación + un albarán por negocio con el código de recogida; máx. 7 días). En la **ficha nativa del pedido** aparece un panel Health Corner con toda la info y el botón de albarán.
- **Liquidaciones**: por negocio y periodo, con **comisiones**, **conciliación con los pagos Redsys** y export **CSV y PDF**. Con "Todos" ves el **resumen de todos los negocios**.

8. RGPD

En **API y registro** → **RGPD**: exportas o borras (anonimizas) los datos de un huésped por **email/DNI**, y ejecutas la **anonimización programada** (o la dejas al cron). Los pedidos se conservan por contabilidad, sin datos personales pasado el plazo.

9. Modo quiosco

Actívalo en *Configuración* → *Avanzado*. Para dejar una **tablet en recepción**: botones grandes, teclado numérico y **reinicio automático** tras inactividad.

10. Problemas frecuentes

- **"No redirige a Redsys" / se pierde el formulario**: re-guarda la clave Redsys en este servidor y regenera el `.htaccess`. Si persiste, revisa `logs/diag.log`.
- **No sale el botón de pagar con tarjeta**: la clave no descifra (mira el semáforo en *Ayuda*). Vuelve a guardarla.

- **La pestaña Health Corner no aparece en el producto:** sube la versión nueva y entra en Configuración; si no, reinstala.
- **Un cambio no se refleja:** es caché (Smarty/opcache). Entrar en Configuración la limpia.

Health Corner — Documentación técnica

Módulo `ecom_healthcorner` (clase `Ecom_Healthcorner`, extiende `PaymentModule`). Estructura **legacy** compatible con PrestaShop **1.7.8.x, 8.0–8.2 y 9.x** (PHP 7.1–8.3). Sin `src/`, sin Composer, sin namespaces, sin overrides, sin `specific_price` de catálogo. Versión **1.2.0**.

1. Estructura

```
ecom_healthcorner.php      Clase principal (global namespace), hooks, config
config.xml
classes/                  Cargadas con require_once (sin autoload Composer)
controllers/admin/        ModuleAdminController (menú propio)
controllers/front/        ModuleFrontController (rutas /hc/...)
views/{css,js,templates,img}
translations/{7 idiomas}/*.xlf
mails/{7 idiomas}/
sql/install.php sql/uninstall.php (SQL inline en PHP, NUNCA install.sql parseado)
docs/ logs/ index.php (en cada dir)
```

Clases (`classes/`)

`EcomHcInstaller` (tablas, entidades, self-heal, upgradeSchema), `EcomHcHotel` (ObjectModel del negocio), `EcomHcHotelContext`, `EcomHcCatalog`, `EcomHcPriceResolver` / `EcomHcPricing` (motor de precios), `EcomHcFront` (layout, i18n por navegador, nonce), `EcomHcCms`, `EcomHcQr` (QR SVG/PNG puro), `EcomHcCheckoutBuilder`, `EcomHcRedsysApi`, `EcomHcRedsysApi`, `EcomHcMailer`, `EcomHcPayments`, `EcomHcHotelPanelAuth`, `EcomHcMachineAuth` (login reponedor), `EcomHcOrders`, `EcomHcSettlement`, `EcomHcMachine` (doble stock), `EcomHcApi` + `EcomHcApiDocs` (API propia), `EcomHcWebservice` (recursos del webservice nativo), `EcomHcCsv`, `EcomHcVenueLock` (cookie de aislamiento), `EcomHcPlanogram`, `EcomHcPickup` (código y estados), `EcomHcDeliveryNote` (albaranes), `EcomHcRgpd`, `EcomHcRestockAlert`, `EcomHcCrypto`, `EcomHcCompat`, `EcomHcLogger`.

Admin controllers

`AdminEcomHc` (padre), Hotels, Products, Prices, Machines, Orders, Settlements, Api (claves + registro + RGPD + docs), Config, **Help** (ayuda), Logs. Menú propio con `id_parent=0` (creación manual de Tabs; `fixMainTabIntegrity()` deduplica por SQL antes de `getAdminLink`).

Front controllers y rutas (`hookModuleRoutes`)

`catalog` `/hc/{slug}`, `product` `/hc/{slug}/p/{id}-{rewrite}`, `cart`, `checkout`, `confirmation` `/hc/{slug}/confirm`, `legal` `/hc/{slug}/legal/{code}`, `panel` `/hc/{slug}/panel`, `track` `/hc/{slug}/track`, `payment` `/hc/{slug}/pay`, `notify` `/hc/notify`, `cron` `/hc/cron`, `reponedor` `/hc/reponedor`, `api` `/hc/api/v1/{recurso}`.

2. Tablas (15)

`ecomhc_hotel`, `ecomhc_hotel cms`, `ecomhc_hotel_user`, `ecomhc_panel_login`, `ecomhc_hotel_product`, `ecomhc_price`, `ecomhc_cart_meta`, `ecomhc_guest_order`, `ecomhc_payment`, `ecomhc_machine`, `ecomhc_machine_stock`, `ecomhc_machine_movement`, `ecomhc_api_key`, `ecomhc_venue_visit`, `ecomhc_machine_user`.

Las de clave compuesta (`hotel_product`, `price`, `machine_stock`) llevan un **id sustituto** `AUTO_INCREMENT` (para ObjectModel/webservice). Columnas añadidas por `upgradeSchema()` (idempotente): `venue_type`, `venue_type_label`, `unit_label_name`, `machine_threshold`, `pickup_code`, `card_token`, `alerted_at`, ids sustitutos.

Instalación: `sql/install.php` es **PHP** que ejecuta cada `CREATE TABLE IF NOT EXISTS` sin abortar al primer fallo. `ensureInstalled()` (self-heal, llamado en cada admin controller) recrea tablas que falten, ejecuta `upgradeSchema()` incondicional, reintenta grupo/zona/carriers/estados y, si cambia la versión, re-registra hooks y tabs. Todo envuelto en try/catch.

3. Motor de precios (híbrido, multi-versión)

- **PS ≥ 8.1:** `hookActionProductPriceCalculation` sobrescribe `$params['price']` por referencia (parámetro NO tipado).
- **1.7.8 / 8.0:** `actionGetProductPropertiesAfter` + filas efímeras de `specific_price` con `id_cart` (limpiadas en `actionValidateOrder`).
- Contexto activo por cookie/slug (`EcomHcHotelContext`); en la notificación (server-to-server, sin cookie) se **activa el hotel a mano** antes de `getOrderTotal` / `validateOrder`.
- Carriers ocultos (`is_module=1`, `need_range=0`, `is_free=1`, sin `addZone`) → invisibles en el checkout normal; en `actionValidateOrder` se fija el carrier real y se ponen portes a 0.

4. Pago

Redsys (propio), alineado a la librería oficial `apiRedsys` : `Ds_MerchantParameters` en **base64 estándar**; firma **3DES-CBC(clave base64-decodificada, IV 8 ceros, zero-padding) + HMAC-SHA256**, salida base64; `Ds_SignatureVersion` = `HMAC_SHA512_V2` (por defecto) o `HMAC_SHA256_V1`. Número de pedido **único** (secuencia `ECOMHC_REDSYS_SEQ`) para evitar SIS0051. `Ds_Merchant_PayMethods` (Bizum `z`, PayPal `p` ...), `Ds_Merchant_ConsumerLanguage`, tokenización COF (`Ds_Merchant_Identifier=REQUIRED`).

Flujo: checkout (POST) → `CheckoutBuilder::prepare` (cliente comercial + address `HC-{id_cart}`) → redirect a `payment` **con id_cart en la URL** (evita el bucle por pérdida del carrito de invitado) → `payment.php` reengancha el carrito desde `cart_meta` y auto-envía el formulario a Redsys → **notificación server-to-server** `notify.php` (valida firma con `hash_equals` normalizado, comprueba `Ds_Amount`, idempotencia con `GET_LOCK`, activa hotel, `validateOrder`, envía correos, guarda token).

Otras formas de pago: `EcomHcPayments` ejecuta el `hookPaymentOptions` de los módulos autorizados; `EcomHcVenueLock` **exime** los módulos de pago y assets estáticos del bloqueo por negocio; el `hookDisplayHeader` redirige `order-confirmation` nativo al `confirmation` del módulo.

5. Vending (doble stock)

Stock de tienda = nativo PS (no se toca). Stock de máquina en `ecomhc_machine_stock` con **libro de movimientos** `ecomhc_machine_movement`. `addMovement()` es **idempotente por** (`id_machine`, `reference`); los movimientos internos (reposición/conteo/ajuste) generan una **referencia única** (PrestaShop guarda `null` como `''` y la clave única solo dejaría un movimiento vacío por máquina → se perdían líneas). `restock()` = conteo real; `alerted_at` para alertas anti-repetición; 3 escenarios Jofemar (A API / B ficheros / C conteo).

6. Aislamiento y seguridad

Layout 100% propio (sin header/footer del tema madre), `noindex,nofollow` + robots.txt (`hookActionAdminMetaBeforeWriteRobotsFile`). `EcomHcVenueLock` (cookie HMAC-firmada) bloquea al visitante en su negocio con excepciones: empleado BO, IPs, rutas técnicas, módulos de pago, assets. Clave Redsys **cifrada** (`EcomHcCrypto`, AES/HMAC sobre `_COOKIE_KEY_`). CSRF por nonce propio en front/panel/reponedor; tokens en formularios admin. API propia con hash-only de claves y `hash_equals`. Webservice: `guest_order` / `cart_meta` / `payments` / `movements` / `venue_visits` en solo-lectura (RGPD/contable). Regla **anti-scan**: ninguna acción externa recorre histórico (ventana ≤24h, dedupe).

7. i18n y correos

`isUsingNewTranslationSystem()=true`. `.tpl`: `{l s='...' d='Modules.Ecomhealthcorner.Shop|Admin'};` PHP: `$this->trans('...', [], 'Modules.Ecomhealthcorner.Admin')`. 7 idiomas (es/en/fr/pt/it/de/ca). Detección de idioma por `Accept-Language` (`EcomHcFront::detectLanguage`). Correos: `order_guest` (con código+QR adjunto+desglose) y `ecomhc_restock`.

8. Configuración (claves `ECOMHC_*`)

Pago/Redsys, `PAYMETHODS`, `TOKENIZE`, `PAYMENT_*`; checkout/email; panel; máquinas/planograma (`MACHINE_MODE`, `MACHINE_THRESHOLD`, `PLANOGRAM_*`, `RESTOCK_EMAIL`, `RESTOCK_WHATSAPP`); API (`API_RATE`, `CRON_TOKEN`); aislamiento (`LOCK_ENABLED`, `BYPASS_IPS`); quiosco (`KIOSK_ENABLED`, `KIOSK_TIMEOUT`); RGPD (`PURGE_MONTHS`); `KILL_SWITCH`, `DEBUG`. Formulario `HelperForm` en pestañas; `getContent()` limpia caché (`clearShopCache`).

Contrato técnico ecom_healthcorner — compatibilidad PS 1.7.8.11 / 8.0-8.2 / 9.1

Resumen accionable de la auditoría del core real (detalle completo con `fichero:línea` en `../././AUDIT_FINDINGS.md`). **Todo lo de aquí está verificado, no supuesto.**

Sintaxis PHP: suelo 7.1

No usar: arrow fn, propiedades tipadas, `??=`, spread en array literal, union types, `match`, promotion, `?->`, named args, `enum`, `readonly`, `str_contains`. Sí: `?Tipo $x=null` (con `? explícito`), `??`, `[$a,$b]=$x`, `catch(A|B $e)`. Castear `(string)/(int)` antes de funciones internas.

Métodos eliminados en 9.1 (Fatal) → sustitutos

No usar	Usar
<code>Controller::ajaxDie(\$j)</code>	<code>\$this->ajaxRender(\$j); exit;</code> o <code>header()+echo+exit</code>
<code>\$this->l()</code> en admin	<code>\$this->trans('x', [], 'Modules.Ecomhealthcorner.Admin')</code>
<code>Tools::displayPrice()</code>	<code>Tools::getContextLocale(\$ctx)->formatPrice(\$p, \$iso)</code>
<code>Tools::link_rewrite()</code>	<code>Tools::str2url()</code>
<code>Tools::jsonEncode/Decode</code>	<code>json_encode / json_decode(\$s, true)</code>
<code>Tools::encrypt/encryptIV</code>	<code>EcomHcCrypto</code> (openssl + <code>COOKIE_KEY</code>)
<code>Order::getOrderByCartId()</code>	<code>(int) Order::getIdByCartId(\$idCart)</code>
<code>Validate::isPasswd</code>	validar longitud + <code>Hashing::hash()</code>
<code>Configuration::getInt()</code>	<code>(int) Configuration::get()</code>
<code>Controller::addJquery()</code>	(jQuery ya está en el front)

Firmas que cambian

- `validateOrder` : 10 params en 1.7.8, 11 en 8.x/9.1 → llamar con **máx 9 posicionales**, 9º = `$customer->secure_key`. Nunca `$shop/$order_reference`.
- `Cart::getProducts()` : 6 params en 9.1, 5 antes → llamar sin posicionales extra.
- `getPriceStatic($id, bool $usetax, ..., ?Context $context)` : pasar bool real y Context|null explícito.

- Errores `validateOrder / Configuration::updateValue : die()` en 1.7.8/8.x (NO capturable) vs `throw` en 9.1 → **blindar antes**, no confiar en try/catch.

Motor de precios (HÍBRIDO — decisión del usuario)

Clase única `EcomHcPriceResolver::resolve()` (fuente de verdad). Adaptadores finos: 1.

actionProductPriceCalculation (solo 8.1+): `function hookActionProductPriceCalculation($params)` sin array ni `&`; `if (isset($params['price'])) { $params['price'] = $nuevo; }` (referencia real). Cubre ficha, listado, carrito, total y order_detail. 2. **actionGetProductPropertiesAfter** (desde 1.7.8): reescribe `$params['product']['price'], ['price_tax_exc'], ['price_amount']` → precio VISIBLE en 1.7.8. 3. **actionCartUpdateQuantityBefore** (las 4): en 1.7.8/8.0, fila efimera en `specific_price` con `id_cart` (scope-carrito, invisible en BO), borrada en `actionValidateOrder`. Precio REAL en carrito/pedido. - Guard estático anti-recursión (`static $inProgress`). No llamar a `getPriceStatic` dentro del hook. - `EcomHcPriceResolver::flushAllCaches() = Product::flushPriceCache() + SpecificPrice::flushCache()` (+ `Product::resetStaticCache()`), invocado en login/logout/cambio de config. - Early return del hook en la 1ª línea sin queries si el contexto HC no está activo (flag en memoria, no cookie).

Carrier oculto

`is_module=1, need_range=0, is_free=1, shipping_method=SHIPPING_METHOD_FREE`, `delay` por idioma, **sin** `addZone()` → excluido del checkout normal en las 4. Guardar id en Configuration + comprobar por `id_reference` antes de crear. Guardar/restaurar `PS_CARRIER_DEFAULT`. La elección de carrier no sobrevive en `$cart->id_carrier` → guardarla en `ecomhc_cart_meta.id_carrier` y fijar el carrier real del pedido en `actionValidateOrder` (`update orders + order_carrier`; en 9.1 también `shipment` si la tabla existe). `uninstall: soft-delete por SQL, no $carrier->delete()`.

Checkout / pedido

Extender `PaymentModule`, `parent::install()`. `hookPaymentOptions` devuelve SIEMPRE `[]` o `[PaymentOption]`. `use PrestaShop\PrestaShop\Core\Payment\PaymentOption;`. Customer: `Customer::customerExists($email,true,true)` antes de add; pass `ServiceLocator::get('\\PrestaShop\\PrestaShop\\Core\\Crypto\\Hashing')->hash($plain); is_guest=0`. Address: `validateFields(false,true)`, DNI con `Address::dniRequired()`, CP con `Country::checkZipCode()`. Guard previo obligatorio (cart cargado, customer/address/carrier>0, `checkQuantities()===true`, idempotencia `Order::getIdByCartId()`). Post-pedido en `actionValidateOrder` + `actionPaymentConfirmation` (no `...After/...Before`). Mails: `mails/es|en/*.html y *.txt`, `Mail::send(..., _PS_MODULE_DIR_. 'ecom_healthcorner/mails/', ...)`, fallback a `en`.

Admin / tabs

`ModuleAdminController` legacy funciona en las 4. `protected $tabs` en constructor (nunca INSERT SQL por `hide_host_mode`). Existencia con `Db::getValue('SELECT id_tab...')`, no `Tab::getIdFromClassName` (NonUniqueResult tumba el BO 9.1). `fixMainTabIntegrity()` dedup por SQL antes de `getAdminLink`. Token:

`getAdminTokenLite / getAdminLink`. AJAX: URL con `getAdminLink(...). '&ajax=1&action=X'`, responder con `helper jsonExit()` propio. `HelperForm file` → pasar clave `'file'` con el input HTML; todo campo de `renderForm` con valor en `fields_value`. `ImageManager::validateUpload($f,$max, ['jpg','jpeg','png','gif'])` (webp no está en 1.7.8).

Front layout propio

En `init()`: `display_header=false; display_footer=false; display_header_javascript=false;`
`parent::init();` → **setMedia NO se ejecuta** → incrustar CSS/JS en el .tpl con `$this->module->getPathUri()`.
`setTemplate('module:ecom_healthcorner/views/templates/front/x.tpl')` sin `{extends}` (el tema classic no está en 9.1). Ficheros de controller front **sin guiones**. AJAX: `$this->ajax=true` en `init`, `ajaxRender()+exit`.
 CSRF: nonce propio en cookie + `hash_equals`. `hookModuleRoutes`: devolver `rule + keywords + params + controller` (omitir `params` descarta la ruta).

Traducciones

`isUsingNewTranslationSystem(){return true;}`. SIEMPRE `d='Modules.Ecomhealthcorner.Shop|Admin'`, NUNCA `mod=`. Ficheros `translations/es-ES/ModulesEcomhealthcornerShop.es-ES.xlf` y `...Admin...` (+ en-US). `|escape:'html':'UTF-8'` siempre, cero `nofilter`, cero `{php} / {displayPrice}`. Precios formateados en PHP y pasados como cadena.

Cifrado / Redsyst / QR

`EcomHcCrypto`: `aes-256-cbc`, clave= `hash('sha256', _COOKIE_KEY_, true)`, IV random por operación, HMAC-SHA256 verificado con `hash_equals` antes de descifrar. NO `PhpEncryption`. `Redsys own`: ORDER 12 chars `str_pad`, amount en céntimos string, firma HMAC_SHA256_V1 (3DES-CBC) o SHA512_V2 seleccionable, comparar con `hash_equals`, notificación server-to-server idempotente. QR: `TCPDF2DBarcode` del core (`class_exists`) → SVG/PNG; fallback generador propio.

DB / instalador

Nunca `LIMIT 1` en `getRow/getValue`. VARCHAR indexado ≤191. `CREATE TABLE IF NOT EXISTS ... ENGINE= .MYSQL_ENGINE`. Migraciones en `addnewfeatures()` con `INFORMATION_SCHEMA`. Config keys `[A-Z0-9_]` ≤254, `Validate::isConfigName` antes de `updateValue`. `index.php` recursivo; `.htaccess` solo en `logs/` y carpetas de subida (nunca anti-PHP en la raíz).

Health Corner — Guía de desarrollo

Guía para mantener y ampliar el módulo. Estructura **legacy** (sin `src/`, sin Composer, sin namespaces).

Objetivo: PrestaShop 1.7.8 / 8.x / 9.x, PHP 7.1–8.3.

1. Instalar / actualizar / desinstalar

- **Instalar:** `install()` registra hooks (`baseHooks` + `actionProductPriceCalculation` solo si $PS \geq 8.1$), crea tablas (`sql/install.php`), entidades y tabs.
 - **Actualizar sin reinstalar:** sube el ZIP y **entra en Configuración**. `getContent()` llama a `addnewfeatures()` y `clearShopCache()`. `ensureInstalled()` (en el constructor de cada admin controller) hace **self-heal**: crea tablas que falten, ejecuta `upgradeSchema()` (idempotente), y si `ECOMHC_HOOKS_V != version` re-registra hooks/tabs. **Sube siempre la version** (en `ecom_healthcorner.php` y `config.xml`) al añadir hooks/rutas/columnas.
 - **Migraciones de columnas:** añádelas en `sql/install.php` (instalación limpia) y en `EcomHcInstaller::upgradeSchema()` (con `columnExists`). Nunca ficheros `upgrade/`.
 - **Desinstalar:** `sql/uninstall.php` (mantener sincronizado con `tableList()`).
-

2. Depuración

- Activa **Depuración** en Configuración → `logs/*.log` por canal (`EcomHcLogger::info/warn/error/exception`). La clave Redsys se enmascara.
 - `logs/diag.log` se escribe **siempre** (sin flag) en puntos críticos del pago (`payment:init ... ready=`, `bounce_to_checkout`) y en la vista de hotel. Útil para el bucle de Redsys.
 - Si "un cambio no se refleja": caché Smarty/opcache → entrar en Configuración la limpia.
-

3. Añadir un formato de planograma

`EcomHcPlanogram::formats()` + un `case` en `build()/fileName()` y un método `toX()`. Nada más cambia. El export está en `AdminEcomHcMachinesController::exportPlanogram()`.

4. Añadir un método de pago

Los módulos de pago externos se ofrecen ejecutando su `hookPaymentOptions` desde `EcomHcPayments::getExternalOptions()`, filtrados por `ECOMHC_PAYMENT_MODULES`. Para que su redirección no la bloquee el aislamiento, `EcomHcVenueLock::isPaymentModule()` los reconoce (lista autorizada + módulos de

pago instalados + heurística por nombre). Redsys propio: parámetros en `EcomHcRedsysApi::buildRequest()`; **la firma debe seguir siendo 3DES+HMAC-SHA256 (librería oficial)** — no usar el método AES documentado en la web (falla SIS0042 en terminales reales).

5. Reglas de firma Redsys (crítico)

`sign()` usa **siempre** `signV1: base64_decode(clave) → 3DES-CBC(order, IV 8 ceros, zero-padding) → HMAC-SHA256(merchantParameters) → base64_encode . Ds_MerchantParameters = base64_encode(json)` (estándar, no base64url). `validateNotification()` normaliza ambas firmas a base64url antes de `hash_equals` (Redsys devuelve la firma en base64url). Número de pedido **único** (secuencia). Ver skills en `C:\dev-tools\prestashop_skills\redsys-master-skills`.

6. API propia (para la máquina)

Rutas `/hc/api/v1/{recurso}` → `EcomHcApi::dispatch()`. Endpoints: `ping`, `hotels`, `machines`, `products`, `stock/shop`, `stock/machine`, `movements` (GET/POST), `restock`. Autenticación `X-HC-API-Key` (hash-only, `hash_equals`), scopes read/write, aislamiento por negocio (`keyHotel()` / `allowsHotel()`; también en `epGetMovements`). Idempotencia por `reference`. Doc viva: `EcomHcApiDocs` (HTML + Markdown, URLs reales) desde *API y registro*.

7. Webservice nativo

`hookAddWebserviceResources` → `EcomHcWebservice::resources()`. 12 recursos `ecomhc_*` con `ObjectModel` + `webserviceParameters`. Asociaciones: `ecomhc_machines` expone `<stock>` (`getWsStock`), `ecomhc_venues` expone `<products>` / `<machines>`. Las tablas de clave compuesta usan su id sustituto.

8. App del reponedor

`controllers/front/reponedor.php` (self-contained: HTML+CSS+JS inline + manifest PWA). Login por Customer (`EcomHcMachineAuth`, tabla `ecomhc_machine_user`, cookie firmada). Endpoints AJAX en el propio controller: `login/logout/hotels/machines/products/stock`. La escritura pasa por `EcomHcMachine::addMovement` (auditable). Alta de reponedores en `AdminEcomHcMachines`.

9. Convenciones

- Cabecera AFL-3.0 en todos los ficheros; `index.php` en cada directorio; `if (!defined('_PS_VERSION_')) exit;`.
- SQL con `pSQL` / `(int)` / `bqSQL`; nunca `LIMIT 1` manual en `getRow` / `getValue`.

- Compatibilidad: `EcomHcCompat` para métodos que cambian entre versiones; `\Context::getContext()` con backslash; `is_callable(['parent','m'])` antes de `parent::`.
 - Smarty: escape obligatorio, sin `nofilter` salvo CSS ya saneado (`EcomHcCms::sanitizeCss`) y HTML de correo controlado.
 - **Extracción/diff de traducciones con script determinista** (regex `.tpl / .php` vs `resname` del `.xlf`), no con agentes.
 - **Multi-agente** para diagnósticos/auditorías (Workflow). La causa raíz del pago se documentó en memoria (`healthcorner-redsys-pago`).
-

10. Puntos abiertos

- Formato real de import de **JSuite/Jofemar** (pendiente del fabricante); el exporter tiene punto de extensión.
- `srcset` real requiere generar varios tamaños de imagen (ahora `loading=lazy` + `home_default`).
- El pago con tarjeta **requiere credenciales Redsys reales** (FUC/terminal/clave) para funcionar en producción.

Health Corner — Documentación de APIs

El módulo expone **dos APIs distintas y complementarias**. Esta es la referencia estática; el back office genera además una versión con las URLs reales de la tienda en **Health Corner → API y registro → Abrir documentación de la API** (HTML imprimible o descarga Markdown), producida por `EcomHcApiDocs`.

	API 1 — Webservice PrestaShop	API 2 — API propia Health Corner
Para qué	Integraciones de gestión: ERP, BI, sincronización de catálogo/precios	La máquina de vending y el software que la controla
URL	<code>https://TU-TIENDA/api/{recurso}</code>	<code>https://TU-TIENDA/hc/api/v1/{recurso}</code>
Autenticación	Clave de webservice de PrestaShop (HTTP Basic, usuario = clave, sin contraseña)	Cabecera <code>X-HC-API-Key</code>
Formato	XML (por defecto) o JSON con <code>?output_format=JSON</code>	JSON siempre
Permisos	Por recurso y por método, en <i>Parámetros avanzados → Webservice</i>	Ámbito <code>read</code> o <code>read,write</code> por clave, y opcionalmente acotada a un negocio
Ventaja	CRUD completo sobre TODAS las tablas del módulo, sin escribir código	Endpoints de negocio (reposición, venta idempotente) listos para la máquina

API 1 — Webservice nativo de PrestaShop

Activación

1. Back office → **Parámetros avanzados** → **Webservice** → activar el servicio.
2. **Añadir nueva clave**: se genera una clave de 32 caracteres.
3. En la lista de permisos aparecen los recursos de Health Corner (los publica el módulo con el hook `addWebserviceResources`). Marcar solo los que necesite la integración.
4. Requiere HTTPS y `.htaccess` regenerado (*Preferencias → SEO y URLs → Generar fichero .htaccess*).

Autenticación

```
curl -u 'TU_CLAVE_WS:' 'https://TU-TIENDA/api/ecomhc_venues?output_format=JSON'
```

La clave va como **usuario** de HTTP Basic; la contraseña se deja vacía (los dos puntos finales son obligatorios).

Sintaxis común

Qué quiero	Petición
Listar ids	<code>GET /api/{recurso}</code>
Listar con todos los campos	<code>GET /api/{recurso}?display=full</code>
Solo algunos campos	<code>GET /api/{recurso}?display=[id,name,active]</code>
Filtrar	<code>GET /api/{recurso}?filter[active]=1</code>
Filtrar por rango	<code>GET /api/{recurso}?filter[id]=[1,10]</code>
Buscar por texto	<code>GET /api/{recurso}?filter[name]=%25Hotel%25</code>
Ordenar	<code>GET /api/{recurso}?sort=[name_ASC]</code>
Paginar	<code>GET /api/{recurso}?limit=20,50</code>
Plantilla de alta	<code>GET /api/{recurso}?schema=blank</code>
Crear	<code>POST /api/{recurso}</code> con el XML de la plantilla
Modificar	<code>PUT /api/{recurso}/{id}</code>
Borrar	<code>DELETE /api/{recurso}/{id}</code>

Recursos publicados por Health Corner

Prefijo de tabla `ps_` a título de ejemplo (usa el real de la tienda).

Recurso	Tabla	Métodos	Descripción
ecomhc_venues	ps_ecomhc_hotel	GET/POST/PUT/DELETE	Negocios (hoteles, posadas, gimnasios...). Todos los campos de marca, precios, servicio, máquina y aislamiento.
ecomhc_venue_cms	ps_ecomhc_hotel_cms	GET/POST/PUT/DELETE	Hojas legales CMS por negocio.
ecomhc_venue_users	ps_ecomhc_hotel_user	GET/POST/PUT/DELETE	Usuarios del panel de cada negocio y sus permisos.
ecomhc_venue_products	ps_ecomhc_hotel_product	GET/POST/PUT/DELETE	Productos asignados a cada negocio, canal (máquina/entrega), destacado, posición.
ecomhc_prices	ps_ecomhc_price	GET/POST/PUT/DELETE	Precio por negocio (modo, %, precio fijo sin impuestos).
ecomhc_cart metas	ps_ecomhc_cart_meta	GET/POST/PUT/DELETE	Datos del huésped asociados al carrito. Contiene datos personales: dar permiso solo a claves restringidas.
ecomhc_venue_orders	ps_ecomhc_guest_order	GET/POST/PUT/DELETE	Pedidos de huésped (extensión del pedido nativo; <code>id_order</code> = pedido de PrestaShop).
ecomhc_payments	ps_ecomhc_payment	solo GET	Intentos de pago Redsys. La escritura la hace solo la notificación firmada del TPV.
ecomhc_machines	ps_ecomhc_machine	GET/POST/PUT/DELETE	Máquinas de vending.
ecomhc_machine_stocks	ps_ecomhc_machine_stock	GET/POST/PUT/DELETE	Stock por canal de cada máquina.

Recurso	Tabla	Métodos	Descripción
ecomhc_machine_movements	ps_ecomhc_machine_movement	GET/POST (no PUT/DELETE)	Libro de movimientos de stock (contable: no se edita ni se borra).
ecomhc_venue_visits	ps_ecomhc_venue_visit	solo GET	Registro de visitas y bloqueos por negocio (auditoría).

Tablas NO publicadas a propósito:

Tabla	Motivo
ps_ecomhc_api_key	Guarda las huellas de las claves de la API propia. Publicarla sería regalar el control de acceso.
ps_ecomhc_panel_login	Intentos de acceso al panel; solo sirve al antifuerza bruta y contiene emails.

Las tablas de clave compuesta (ecomhc_hotel_product, ecomhc_price, ecomhc_machine_stock) llevan un id sustituto AUTO_INCREMENT (id_hotel_product, id_price, id_machine_stock); el webservice y ObjectModel exigen clave primaria simple. La clave compuesta original se conserva, así que la lógica del módulo no cambia.

Ejemplos

```
# Listar negocios con todos sus campos, en JSON
curl -u 'TU_CLAVE_WS:' 'https://TU-TIENDA/api/ecomhc_venues?display=full&output_format=JSON'

# Productos del negocio 3 que están en la máquina
curl -u 'TU_CLAVE_WS:' 'https://TU-TIENDA/api/ecomhc_venue_products?display=full&filter[id_hotel]=3&filter[in_machine]=1&output_format=JSON'

# Stock actual de la máquina 1
curl -u 'TU_CLAVE_WS:' 'https://TU-TIENDA/api/ecomhc_machine_stocks?display=full&filter[id_machine]=1&output_format=JSON'

# Pedidos de huésped del negocio 3 en un rango de fechas
curl -u 'TU_CLAVE_WS:' 'https://TU-TIENDA/api/ecomhc_venue_orders?display=full&filter[id_hotel]=3&filter[date_add]=[2026-01-01,2026-12-31]&date=1&output_format=JSON'
```

API 2 — API propia de Health Corner (máquina de vending)

Por qué existe

El webservice de PrestaShop es CRUD sobre tablas: para registrar una venta habría que leer el stock, calcularlo y escribirlo, con riesgo de duplicar si la máquina reintentó. Esta API expone **operaciones de negocio** ya resueltas e **idempotentes**, que es lo que necesita un dispositivo con conectividad intermitente.

URL base

```
https://TU-TIENDA/hc/api/v1/{recurso}
```

Autenticación

Cabecera `X-HC-API-Key` con la clave generada en **Health Corner → API y registro**. La clave se muestra **una sola vez**: en la base de datos solo se guarda su huella SHA-256. Si se pierde, se revoca y se crea otra.

Cada clave tiene:

- **Ámbito**: `read` (consulta) o `read,write` (además registra ventas y reposiciones).
- **Negocio**: opcionalmente acotada a uno solo. Una clave acotada no ve datos de otros negocios aunque los pida.

Reglas comunes

- Solo HTTPS; las peticiones en claro se rechazan.
- Respuesta siempre JSON con `Content-Type: application/json`.
- Cabecera `X-Robots-Tag: noindex` en todas las respuestas.
- Límite de peticiones por minuto y clave: configurable (`ECOMHC_API_RATE` , default 120). Al superarlo, HTTP 429.
- **Idempotencia**: toda escritura acepta `reference`. Reenviar la misma `reference` para la misma máquina NO duplica el movimiento: devuelve `duplicate_reference: true`. Es la garantía frente a reintentos por corte de red.

Formato de respuesta

```
{ "ok": true, "data": { }, "meta": { "count": 0 } }
```

Errores:

```
{ "ok": false, "error": { "code": "forbidden", "message": "La clave no tiene permiso de escritura" } }
```

HTTP	Código	Cuándo
400	<code>bad_request</code>	Falta un parámetro obligatorio o tiene un valor inválido
401	<code>unauthorized</code>	Falta <code>X-HC-API-Key</code> , o la clave no existe / está revocada
403	<code>forbidden</code>	La clave no tiene ámbito <code>write</code> , o pide datos de otro negocio
404	<code>not_found</code>	Recurso o registro inexistente
429	<code>rate_limited</code>	Superado el límite de peticiones por minuto
500	<code>server_error</code>	Error interno (queda en los logs del módulo)

Endpoints

Método	Ruta	Ámbito	Descripción
GET	<code>ping</code>	read	Comprobación de vida. Devuelve <code>pong</code> , <code>time</code> , <code>machine_mode</code> (A/B/C), <code>write</code> .
GET	<code>hotels</code>	read	Negocios visibles para la clave. Params: <code>active</code> .
GET	<code>machines</code>	read	Máquinas del negocio. Params: <code>id_hotel</code> , <code>code</code> . Devuelve <code>id_machine</code> ...
GET	<code>products</code>	read	Catálogo con precio ya resuelto para el negocio (con y sin IVA). Params: <code>id_hotel</code> , <code>channel</code> (machine/delivery/all), <code>limit</code> .
GET	<code>stock/shop</code>	read	Stock de la TIENDA (nativo PS), que abastece la entrega. Params: <code>id_hotel</code> , <code>id_product</code> .
GET	<code>stock/machine</code>	read	Stock DENTRO de la máquina por canal. Params: <code>id_machine</code> , <code>only_in_stock</code> .
GET	<code>movements</code>	read	Libro de movimientos. Params: <code>id_machine</code> , <code>from</code> , <code>to</code> , <code>limit</code> .
POST	<code>movements</code>	write	Registra una venta (o merma) de máquina. Params: <code>id_machine</code> , <code>id_product</code> , <code>type</code> (out/in/waste/count), <code>qty</code> , <code>reference</code> (idempotencia), <code>note</code> .
POST	<code>restock</code>	write	Reposición con conteo real (escenario C). Params: <code>id_machine</code> , <code>lines[]</code> = { <code>id_product</code> , <code>channel</code> , <code>counted</code> , <code>loaded</code> } .

Ejemplos

```
# 1. Comprobar conectividad y clave
curl -H 'X-HC-API-Key: hc_xxx' 'https://TU-TIENDA/hc/api/v1/ping'

# 2. Localizar la máquina por su código
curl -H 'X-HC-API-Key: hc_xxx' 'https://TU-TIENDA/hc/api/v1/machines?code=VM-LOBBY-01'

# 3. Descargar el catálogo con precios resueltos para ese negocio
curl -H 'X-HC-API-Key: hc_xxx' 'https://TU-TIENDA/hc/api/v1/products?id_hotel=3&channel=machine'

# 4. Registrar una venta (idempotente por reference)
curl -X POST -H 'X-HC-API-Key: hc_xxx' -H 'Content-Type: application/json' \
  -d '{"id_machine":1,"id_product":42,"type":"out","qty":1,"reference":"VM1-2026-000123"}' \
  'https://TU-TIENDA/hc/api/v1/movements'

# 5. Reposición con conteo real
curl -X POST -H 'X-HC-API-Key: hc_xxx' -H 'Content-Type: application/json' \
  -d '{"id_machine":1,"lines":[{"id_product":42,"channel":"11","counted":2,"loaded":8}]}' \
  'https://TU-TIENDA/hc/api/v1/restock'
```

Cómo integrar la máquina según lo que permita JSuite

A fecha de hoy Jofemar **no ha confirmado** qué expone la Vision EsPlus / JSuite. La integración está preparada para los tres escenarios y se elige en **Configuración → Máquinas y planograma → Modo de integración**:

Modo	Cómo funciona	Qué hay que hacer
A — Telemetría por API	JSuite (o un middleware) llama a <code>POST /movements</code> en cada venta	Crear una clave <code>write</code> y dársela al integrador. Stock exacto al minuto.
B — Intercambio de ficheros	JSuite exporta ventas a fichero y se importan periódicamente	Import CSV desde <i>Productos</i> , o un script que llame a <code>POST /movements</code> línea a línea (la <code>reference</code> evita duplicar en reimportaciones).
C — Conteo manual (por defecto)	El reponedor cuenta lo que queda en cada visita y el módulo deduce las ventas	Nada que integrar. El stock se marca <code>estimated</code> y el umbral de seguridad evita anunciar lo agotado.

En los tres casos el planograma se exporta desde **Health Corner → Máquinas vending → Planograma** (formatos: genérico, JSuite/Jofemar, telemetría).

Seguridad

- Las claves de la API propia se guardan **solo como huella** SHA-256 con sal del `_COOKIE_KEY_` de la tienda: ni el administrador puede recuperarlas.
- La comparación de claves usa `hash_equals` (tiempo constante).
- Toda clave se revoca al instante desde el back office; la revocación es efectiva en la siguiente petición.
- Las claves acotadas a un negocio no pueden leer ni escribir datos de otro, aunque manipulen los parámetros.
- La API propia nunca devuelve datos personales de huéspedes: para eso está el webservice, con sus permisos por recurso.